

EL Explicator - Interactive Repair Mode Evaluation

July 2026

Contents

1 Experiments	2
1.1 Evaluating Time Consumption	2
1.1.1 Defect Axiom Selection	2
1.1.2 Interesting Axiom Ontology Generation	2
1.1.3 Running the Repair Process	2
1.1.4 Evaluating “not sure” Options	3
1.1.5 Class Hierarchy Construction	3
1.2 Defect Axioms Selection	3
1.2.1 Interesting Axiom Ontology Generation	4
1.2.2 Normalization of the Ontology	4
1.2.3 ABox Generation	4
1.2.4 Evaluating Options	5
1.2.5 Cost Computing Function	5

1 Experiments

The evaluation of the “not sure” options was conducted in two different ways. First, the evaluation focused on the time consumption of the repair process, measuring how long the process took to complete under each decision strategy. Second, it evaluated the quality of information provided by the “not sure” options by comparing the resulting repairs against an ABox ontology using a defined cost function.

The dataset used in the experiments consists of a collection of EL ontologies made available as part of the experiment resources for Explaining Non-Entailment by Model Transformation for the Description Logic EL archived on Zenodo at <https://zenodo.org/records/7064514>.

1.1 Evaluating Time Consumption

To evaluate the time efficiency of computing results for the “not sure” options in the repair process, a benchmarking experiment was conducted on the dataset of ontologies. For each ontology file listed in the dataset, the ontology is first loaded, a defect axiom is selected and an ontology consisting of interesting axioms is generated. The justification information such as total number of axioms in the ontology and defect module, number of justifications, maximum justification size and maximum common justification axioms are computed to provide insights on the repair task. The repair process is then conducted for the ontology. It is run 3 iterations for each “not sure” option. A CSV file is used to keep records of the experiment results.

1.1.1 Defect Axiom Selection

The defect axiom selection process involves fetching all classes in the ontology signature and randomly selecting 20 of them. For each selected class, all inferred subclasses are obtained, excluding the $\perp$ class and the respective `SubClassOf` axioms are created. These axioms are collected in a list, which defaults to the explicitly stated `SubClassOf` axioms in the ontology if no inferred axioms are found. A defect axiom is then randomly picked from this list.

1.1.2 Interesting Axiom Ontology Generation

Selecting an interesting axiom follows a similar approach as defect axiom selection, except it maintains two sets of `SubClassOf` axioms: one containing non-explicit axioms which do not have $\perp$ as the subclass and another containing all remaining inferred axioms. It also checks beforehand if the inferred axiom is the defect axiom and excludes it from these sets if true. If the first set is non-empty, an axiom is randomly selected from it, else, an axiom is selected from the second set. If both sets are empty, a `SubClassOf` axiom stated in the ontology is randomly chosen. A new OWL ontology consisting of only this selected axiom is created and saved, which serves as the interesting axiom ontology for the repair process.

1.1.3 Running the Repair Process

For the benchmarking experiment, each ontology with its defect axiom and interesting axiom ontology is processed through 3 iteration for each not sure option. The jar file is run with the necessary

arguments, including ELK as the reasoner, while user input is simulated via a text file corresponding to the “not sure” option. The input sequence simulates the user selecting “not sure” for the first justification axiom, followed by the appropriate option number and then exiting the repair process. The time taken for the entire process is measured in milliseconds, with a 90 seconds timeout applied. If the process completes within the timeout, the runtime is logged. Otherwise, timeout is indicated with “Timeout”. After completing all 3 runs for each option, the average runtime, standard deviation and standard deviation percentage are computed and recorded in the benchmark results.

The entire benchmarking process is automated using a bash script, which specifies the timeout and number of iterations and executes the Java benchmarking class. The benchmarking class maintains the current state of the benchmark across runs, including the ontology being processed, iteration number, any recorded runtimes and status. In the event of an `OutOfMemoryError`, the status is “Failure”, otherwise “Success”. If the bash script detects a failure status, it reruns the benchmarking class with a flag to indicate it is a rerun. Then, the benchmarking process restarts from the state where it exited.

1.1.4 Evaluating “not sure” Options

The objective of this experiment is to analyze the quality of information provided by the “not sure” options during the repair process. The repair obtained by making decisions based on the information supplied by the not sure options for each justification axiom is evaluated against an ABox ontology.

The evaluation of the “not sure” options is performed once on the original ontologies from the dataset and once for the normalized versions of them. For each ontology in the dataset, the ontology is loaded, up to 10 defect axioms are selected, and depending on the case the considered, input ontology is either the normalized version or the original version of the ontology. The class hierarchies for the original and input ontology are constructed. Then for each defect in the defect set, an important axioms ontology is generated and an ABox ontology is created from the original and input ontology. The repair process is executed once for each of the “not sure” options until a repair is reached. The options evaluated are: Entailment Probability, Class Hierarchy Difference, Hamming Distance, Mix and User. The cost of the resulting repairs from following each of the options are computed and the results are reported in a CSV file. Additionally, to keep track of the decisions made during each repair process, the corresponding justification axiom–answer pairs are logged in a JSON file.

1.1.5 Class Hierarchy Construction

In order to avoid redundant computation of the class hierarchy, once the ontology is loaded, an object representing its class hierarchy is constructed and recorded. For each class in the ontology signature, a mapping to its inferred subclasses and superclasses are recorded in the maps `subClasses` and `superClasses` respectively.

1.2 Defect Axioms Selection

Defect axioms are selected based on the structural influence of classes within the ontology. The influence is determined by the extent to which classes participate in `SubClassOf` and `EquivalentClasses` axioms. For each such axiom, the involved classes are identified and the number of axioms in which each class appears is recorded as the class connectivity score. While iterating through the `SubClassOf` axioms, the list of axioms involving only atomic concepts are also recorded as simple subclass axioms. Then, 10 classes with the highest connectivity scores and also involved in a simple

subclass axiom are selected as the candidate classes. For each of the candidate classes, all inferred subclasses excluding the $\perp$ class are retrieved and the subclass with the highest connectivity score is selected. The corresponding subclass axiom is chosen as one of the defects if it hasn't already been recorded as a defect, else it checks for the next subclass. If no inferred subclasses exist for the candidate class, an inferred superclass with highest connectivity score is instead identified and the corresponding subclass axiom is selected as one of the defects if it isn't already recorded as a defect. The original ontology and its precomputed class hierarchy mapping is used to fetch the inferred subclasses and superclasses, and perform defects selection.

1.2.1 Interesting Axiom Ontology Generation

The interesting axiom ontology generation follows the same procedure as that used in the time-consumption benchmarking experiment but instead of using the reasoner to compute the subclasses, it uses the precomputed class hierarchy mapping of the original ontology to get all the inferred subclasses for the classes in signature.

1.2.2 Normalization of the Ontology

After selection of defect axiom and interesting axiom, a normalized version of the ontology is created by processing all **SubClassOf** axioms from the original ontology. For each axiom the superclass expression is examined. If the superclass expression is atomic, it is reused directly. If it is anonymous, the normalization process first checks whether this complex expression has already been assigned an atomic concept previously. If so, the previously introduced atomic concept is reused. Otherwise, a new atomic concept is created.

To create a new atomic concept, the expression is recursively unfolded to produce a syntactic summary that mirrors its nested structure. For an instance of existential restriction, the IRI is constructed as **Exists** + the IRI short form of the object property + the recursively obtained summary of the filler. If the expression is an instance of object intersection, then the summaries of the operands are recursively computed and concatenated with “_&_”.

For a newly introduced atomic concept, an **EquivalentClasses** axiom is added that defines it in terms of a normalized version of the original complex expression. Normalization of complex expressions is performed recursively where in the case of existential restriction, the filler is replaced by an atomic concept and in case of object intersection, each operand is replaced by their atomic concepts. The fillers and operands are also recursively normalized and this process is repeated until only atomic concepts remain.

Finally, the original **SubClassOf** axiom is replaced with a new axiom in which the superclass is the atomic concept corresponding to the original superclass expression, while the subclass expression remains unchanged.

1.2.3 ABox Generation

The ABox is generated based on the class hierarchy of the input ontology and hence the respective precomputed class hierarchy mapping is used to get the subclass and superclass mappings. First, all the leaf classes are identified, i.e., classes that do not have any subclasses. From the set of these leaf classes, only 80% are randomly selected and used for instance generation.

For each selected leaf class, a random number of individuals, between 1 to 5 are created and are asserted to this class. The instance assertions are then propagated along the class hierarchy by identifying the superclasses via the superclass mapping excluding any unsatisfiable superclasses.

To ensure that the ABox does not support the entailment of the defect axiom, a counterexample is added. If the defect is a `SubClassOf` axiom, a new individual is asserted only to the subclass and not to the superclass. If the defect is an `EquivalentClasses` axiom, the individual is asserted to only one of the equivalent operands. If it is a `DisjointClasses` axiom, the instance is asserted to all operands. These assertions are propagated through the class hierarchy while avoiding satisfaction of the defect axiom.

1.2.4 Evaluating Options

For each input ontology, the repair process is run separately for each option, namely Entailment Probability, Class Hierarchy Difference, Hamming Distance, User and Mix. For each justification axiom, the selected decision depends on the evaluation option under consideration. Under the entailment probability option, the decision corresponding to the higher total entailment probability is selected. In case of same probabilities, “yes” is answered by default. For the class hierarchy difference option, the ontologies resulting from both possible decisions are evaluated against the generated ABox ontology using the cost function and the decision yielding the lower cost is chosen. The Hamming distance option selects the decision that minimizes the Hamming distance. In the user option, decisions are made randomly without relying on the information provided by the “not sure” options. The probability of answering “yes” is initially set to 75% and if no repair is reachable, this probability is gradually reduced until either a repair is found or the probability reaches zero. The mix option computes all three “not sure” options and selects the decision supported by the majority options.

Once a maximal repair is detected, the repair is saved. The cost for this repair is computed and recorded as part of the experimental results. All decisions made during the repair process are logged along with the ontology name, evaluation option used, cost of input ontology, cost of repair and defect axiom in a JSON file. In cases where only single justification exists and user decision is not required, the cost is recorded as -1.

1.2.5 Cost Computing Function

The cost of an ontology is computed against an ABox ontology. the cost is determined based on the effort required to obtain relevant results in the ontology. The following cost function is used to determine the quality of an ontology O with set of concepts C :

$$Cost(O) = \sum_{c \in C} r_{instances}(c) * cost(c)$$

$$cost(c) = cost(parent(c)) + (k_d * depth(c) + k_b * breadth(c))$$

Here, $depth(c)$ is the depth of the class c in the class hierarchy, $breadth(c)$ is the number of siblings of c in the class hierarchy and k_b and k_d are weighted constants for depth and breadth respectively. $r_{instances}(c)$ represents the number of instances classified under concept c in the ABox.

A concept might occur at multiple depths of the hierarchy. The maximum depth is used for computing its cost and the parent associated with this depth is used to also obtain the breadth of the concept. The weighted constants used for depth and breadth in this experiment are 0.5 and 0.3 respectively.